- 稳燃宝
  - 一、数学建模
    - 1. 脱水实验与典型垃圾组分的数学模型
    - 2. 焚烧炉的静态代理模型
    - 3. 焚烧炉的动态代理模型
    - 4. 焚烧炉的稳定工作区间
      - (1) 最低温度约束
      - (2) 平均温度下限约束
      - (3) 平均温度上限约束
      - (4) 最高温度约束
      - (5) 稳态参考含水率区间
      - (6) 稳态工作点
      - (7) 稳态参考平均温度区间
    - 5. 预热炉设计与数学模型
      - (1) 炉型设计结构
      - (2) 垃圾传热模型
        - 垃圾处理过程
        - 垃圾物性
      - (3) 烟气传热模型
        - 可用烟气资源
        - 烟气换热系数
        - 烟气物性
      - (4) 预热炉的动态模型
        - 前端纯延迟
        - 炉内混合的一阶等效
        - 预热炉动态壳
    - 6. 预热炉能耗计算
      - (1) 热负荷
      - (2) 热平衡约束
      - （3）单位时间能耗
    - 7. 执行器的动态模型
  - 二、控制策略与 Python 实现
    - 1. 控制策略的总体思路
    - 2. 外层控制器（Controller）
      - （1）焚烧炉三路反馈量
      - （2）等效含水率状态估计
      - （3）PID 控制律与原始目标请求
      - （4）积分抗饱和与积分分离
    - 3. 治理器（Governor）
      - （1）动态响应因子
      - （2）由当前测量量反推允许目标带
      - （3）稳态参考带的并入与动态带平滑
    - 4. 转译器 A（Transcriber A）
      - （1）由清洗器（cleanser）生成入口等效性质
      - （2）前端延迟离散实现
      - （3）一阶混合槽位更新
      - （4）资源边界与静态请求组装
    - 5. 内层优化器（Optimizer）
      - （1）输入字段与语义
      - （2）决策变量
      - （3）目标带、物理可实现带与目标投影
      - （4）热工基础函数
      - （5）约束条件的完整结构
      - （6）三阶段词典序多目标求解
      - （7）初值种子与数值求解设置
      - （8）输出字段与诊断量

- （9）离线制表与在线查表
- 6. 转译器 B（Transcriber B）
  - （1）命令变化率限制
  - （2）设备基本边界
  - （3）烟气资源修正
  - （4）补热缺口计算
- 7. 闭环动态过程的完整说明
  - 当焚烧炉偏热时
  - 当焚烧炉偏冷时
- 8. 在线实现与时间尺度
  - （1）测量层
  - （2）外层控制层
  - （3）转译与优化层
  - （4）执行器层
- 三、下一步计划

稳燃宝

稳燃宝（FlameGuard）是一是一种基于非线性多目标规划和改进 PID 的垃圾焚烧厂预热炉控制系统。

控制的基本目标是：

> 利用焚烧炉可提供的烟气余热，对高含水率厨余垃圾进行预热与脱水，并通过分层控制结构，使进入主焚烧炉的垃圾含水率维持在适合长期稳定燃烧、又能尽可能降低能耗的区间内。

本控制系统基于如下工程设计问题：设计一种垃圾焚烧预热炉，利用主垃圾焚烧炉产生的高温烟气余热，对高含水率厨余垃圾进行预热脱水，再将预热后的垃圾送入主焚烧炉，以改善主焚烧炉的燃烧状态，实现炉温稳定、降低辅助燃料消耗并减少污染物排放。这种预热炉需要一个稳定、高效的控制方案，控制其输出的垃圾含水量，进而控制主焚烧炉的燃烧状态。

本控制系统的主要能力包括垃圾组分识别、目标治理、静态运筹学优化、与控制器修正。

一、数学建模

1. 脱水实验与典型垃圾组分的数学模型

我们抽取了六种典型的不同成分的高含水厨余垃圾（菜叶、西瓜皮、橙子皮、肉、杂项混合、米饭），将这六种垃圾混合成为混合垃圾进行预热脱水。经过实验，我们测定了这六种成分垃圾的含水率、干燥动力学特征（每种垃圾在 175°C 下降低到 20% 含水率时所需的时间）、温度敏感性特征（升高每单位摄氏度时，垃圾降低到 20% 含水率所需时间的增加量，一般取负值）。

根据实验数据，可将不同组分垃圾的参数制表如下：

类别	A 初始含水率(%)	B 干燥动力学特征	C 温度敏感性
菜叶	94.8	12.1	-0.132
西瓜皮	94.8	17.7	-0.251
橙子皮	81.7	15.3	-0.189
肉	44.2	11.5	-0.216
杂项混合	77.3	16.3	-0.210
米饭	61.1	15.8	-0.243

将六类垃圾编号为：

- x_1：菜叶

- x_2：西瓜皮
- x_3：橙子皮
- x_4：肉
- x_5：杂项混合
- x_6：米饭

并约束：

$$\sum_{i=1}^{6} x_i = 1, \qquad x_i \geq 0$$

将实验中测定得到的三类原始性质按湿基小数和分钟制统一为：

$$\omega_i = \{0.948, 0.948, 0.817, 0.442, 0.773, 0.611\}$$

$$t_{\mathrm{ref},i} = \{12.1, 17.7, 15.3, 11.5, 16.3, 15.8\}\ \mathrm{min}$$

$$s_i = \{-0.132, -0.251, -0.189, -0.216, -0.210, -0.243\}\ \mathrm{min}/^{\circ}\mathrm{C}$$

其中：

- ω_i 表示第 i 类垃圾的初始湿基含水率；
- $t_{\mathrm{ref},i}$ 表示第 i 类垃圾在参考温度 175°C 下达到参考脱水水平所需的时间；
- s_i 表示温度敏感性斜率，取负值意味着温度提高时，达到同一脱水水平所需时间减少。

对混合垃圾，我们有如下基本假设：

1. 各组分垃圾均匀混合；
2. 混合后的等效性质可视为各组分性质的线性组合；
3. 在当前控制温区内，温度敏感性斜率可近似视为常数。

于是可定义混合垃圾的等效性质如下。

混合初始含水率：

$$\omega_0(x) = 0.948x_1 + 0.948x_2 + 0.817x_3 + 0.442x_4 + 0.773x_5 + 0.611x_6$$

混合参考干燥时间：

$$t_{\mathrm{ref}}(x) = 12.1x_1 + 17.7x_2 + 15.3x_3 + 11.5x_4 + 16.3x_5 + 15.8x_6$$

混合温度敏感性斜率：

$$s(x) = -0.132x_1 - 0.251x_2 - 0.189x_3 - 0.216x_4 - 0.210x_5 - 0.243x_6$$

混合垃圾在等效平均干燥温度 T_m 下达到 20% 含水率的动力学时间为：

$$\tau_{20}(T_m, x) = t_{\mathrm{ref}}(x) + s(x)(T_m - 175)$$

该公式是一个参考动力学基准，在后续模型中用于推导“达到任意目标含水率所需时间”的近似表达式。

2. 焚烧炉的静态代理模型

我们在 COMSOL Multiphysics 中基于典型的乡镇级小型垃圾焚烧厂主焚烧炉设计图纸进行了三维建模，并在稳态工况下研究“进入主焚烧炉的垃圾含水率”与焚烧状态之间的关系。经过参数扫描与线性回归，可将主焚烧炉的稳态燃烧状态写成如下代理模型。

本模型统一规定：

- w：进入主焚烧炉的垃圾含水率，单位为 %；
- $\omega = w/100$：同一含水率对应的湿基小数形式；
- 所有由 COMSOL 稳态回归得到的焚烧炉代理方程均使用 w；
- 所有预热炉热负荷、蒸发量、动力学时间缩放等热工计算均使用 ω。

基于该记号，主焚烧炉静态代理模型如下。

烟囱出口平均温度：

$$T_{\text{stack}}(w) = -14.412237w + 1423.472316$$

烟囱出口平均速度：

$$v_{\text{stack}}(w) = -0.215310w + 25.332842$$

燃烧核心区域平均温度：

$$T_{\text{avg}}(w) = -13.109632w + 1294.871365$$

燃烧面温度标准差：

$$\sigma(w) = -0.189997w + 18.331239$$

燃烧面最低温度：

$$T_{\min}(w) = -13.422320w + 1330.692379$$

燃烧面最高温度：

$$T_{\max}(w) = -16.072025w + 1589.019616$$

这些关系共同构成主焚烧炉的静态代理模型，用于把“进入主焚烧炉的垃圾含水率”映射到“主焚烧炉稳态燃烧状态”。其中：

- $T_{\text{avg}}(w)$、$T_{\text{stack}}(w)$、$v_{\text{stack}}(w)$ 是当前工程中最重要的三条可观测稳态代理关系；
- $T_{\min}(w)$ 与 $T_{\max}(w)$ 主要用于定义稳态安全边界；
- $\sigma(w)$ 主要用于描述燃烧均匀性，并在静态优化的次级目标中使用。

3. 焚烧炉的动态代理模型

稳态代理模型只回答“当进入焚烧炉的垃圾含水率长期稳定在某一数值附近时，焚烧炉最终会稳定在什么燃烧状态”。但在真实运行中，进入主焚烧炉的垃圾含水率变化并不会立刻反映为炉温变化，焚烧炉内部还存在输送延迟、炉壁热容以及炉内垃圾混合等动态过程。因此，数学建模中还需要补充焚烧炉的动态代理模型。

当前项目中，将进入主焚烧炉的垃圾含水率 $w_b(t)$ 视为焚烧炉动态对象的输入。焚烧炉可测输出为三类量：

$$y_f(t) = \begin{bmatrix} T_{\text{avg}}(t) \\ T_{\text{stack}}(t) \\ v_{\text{stack}}(t) \end{bmatrix}$$

其中：

- $T_{\text{avg}}(t)$ 为燃烧核心区域平均温度；
- $T_{\text{stack}}(t)$ 为烟囱出口平均温度；
- $v_{\text{stack}}(t)$ 为烟囱出口平均速度。

根据当前 COMSOL 瞬态仿真，三路可测输出共享相同的动态骨架：

1. 一个纯延迟环节，表示预热炉出口垃圾进入主焚烧炉的输送延迟；
2. 一个快一阶环节，表示炉壁热容带来的瞬态迟滞；
3. 一个慢一阶环节，表示新进入垃圾与炉内已有垃圾混合后才逐步改变焚烧状态的过程。

记：

- $\tau_f = 5$ s：焚烧炉前端纯延迟；
- $\tau_{f1} = 0.223$ s：炉壁热容对应的快时间常数；
- $\tau_{f2} = 75.412$ s：炉内垃圾混合对应的慢时间常数。

则对围绕稳态工作点的小扰动变量，可将焚烧炉动态代理模型统一写成：

$$\Delta y_f(s) = G_f(s)\,\Delta w_b(s)$$

其中

$$G_f(s) = \frac{e^{-\tau_f s}}{(\tau_{f1}s+1)(\tau_{f2}s+1)} \begin{bmatrix} K_{\text{avg}} \\ K_{\text{stack},T} \\ K_{\text{stack},v} \end{bmatrix}$$

各通道的静态增益取自上一节的稳态线性回归斜率，即：

$$K_{\text{avg}} = -13.109632$$

$$K_{\text{stack},T} = -14.412237$$

$$K_{\text{stack},v} = -0.215310$$

于是三条具体动态通道分别为：

$$G_{\text{avg}}(s) = \frac{-13.109632\, e^{-5s}}{(0.223s+1)(75.412s+1)}$$

$$G_{\text{stack},T}(s) = \frac{-14.412237\, e^{-5s}}{(0.223s+1)(75.412s+1)}$$

$$G_{\text{stack},v}(s) = \frac{-0.215310\, e^{-5s}}{(0.223s+1)(75.412s+1)}$$

该模型的物理意义如下：

- 当进入焚烧炉的垃圾含水率升高时，燃烧核心区域平均温度下降、烟囱出口温度下降、烟囱出口平均速度也随之下降；
- 这种影响不会立刻出现，而是要先经历前端输送延迟；
- 延迟之后，炉壁热容使得焚烧炉对扰动的最初响应仍然存在一个快迟滞；
- 随后，新进入垃圾与炉内已有垃圾的混合过程决定了主导时间尺度，因此整体对象在工程上是一个明显的慢系统。

4. 焚烧炉的稳定工作区间

根据主焚烧炉长期稳定燃烧的工程要求，在稳态参考分析中采用如下约束：

1. 燃烧面最低温度不得低于 850°C；
2. 燃烧区平均温度不得低于 850°C；
3. 燃烧区平均温度不得高于 1100°C；
4. 燃烧面最高温度不得高于 1100°C；
5. 在满足上述硬约束的前提下，温度标准差越小越好。

(1) 最低温度约束

由

$$T_{\min}(w) \geq 850$$

即

$$-13.422320w + 1330.692379 \geq 850$$

可得：

$$w \leq \frac{1330.692379 - 850}{13.422320} = 35.81$$

(2) 平均温度下限约束

由

$$T_{\text{avg}}(w) \geq 850$$

即

$$-13.109632w + 1294.871365 \geq 850$$

可得：

$$w \leq \frac{1294.871365 - 850}{13.109632} = 33.93$$

(3) 平均温度上限约束

由

$$T_{\text{avg}}(w) \leq 1100$$

即

$$-13.109632w + 1294.871365 \leq 1100$$

可得：

$$w \geq \frac{1294.871365 - 1100}{13.109632} = 14.86$$

(4) 最高温度约束

由

$$T_{\max}(w) \leq 1100$$

即

$$-16.072025w + 1589.019616 \leq 1100$$

可得：

$$w \geq \frac{1589.019616 - 1100}{16.072025} = 30.43$$

(5) 稳态参考含水率区间

综合上述稳态硬约束，真正起主导作用的是：

- 平均温度下限约束给出的上界；
- 最高温度约束给出的下界。

因此，主焚烧炉长期稳定燃烧的稳态参考含水率区间为：

$$30.43\% \leq w \leq 33.93\%$$

对应湿基小数区间为：

$$0.30427 \leq \omega \leq 0.33935$$

这个区间描述的是主焚烧炉在稳态下长期适宜运行的参考带，后文记为稳态参考带 W_{ss}。

(6) 稳态工作点

在当前工程口径下，稳态工作点取稳态参考含水率区间的中点，因此定义：

$$w_{\text{ref}} = \frac{30.43 + 33.93}{2} = 32.18$$

对应湿基小数形式：

$$\omega_{\text{ref}} = 0.3218$$

将 w_{ref} 代入平均温度代理模型，可得稳态目标平均炉温：

$$T_{\text{set}} = T_{\text{avg}}(w_{\text{ref}}) = -13.109632 \times 32.18 + 1294.871365 = 872.99^{\circ}\text{C}$$

因此，当前系统的稳态工作点可记为：

$$(w_{\text{ref}},\ \omega_{\text{ref}},\ T_{\text{set}}) = (32.18,\ 0.3218,\ 872.99^{\circ}\text{C})$$

(7) 稳态参考平均温度区间

将稳态参考含水率区间两端分别代入平均温度代理模型，可得对应的平均温度参考区间：

$$T_{\text{avg}}(30.43) = 895.99^{\circ}\text{C}$$

$$T_{\text{avg}}(33.93) = 850.00^{\circ}\text{C}$$

因此，稳态参考平均温度区间为：

$$850.00^{\circ}\text{C} \leq T_{\text{avg}} \leq 895.99^{\circ}\text{C}$$

5. 预热炉设计与数学模型

(1) 炉型设计结构

预热炉设备原型取卧式间接回转滚筒预热脱水炉，并联设置设外夹套与内置导烟管两级传热面，烟气在进入焚烧炉烟囱前引出，经过主烟道分流后，所需烟气进入预热炉的外夹套与内置导烟管。这种设计是为了满足连续进料、垃圾与烟气不接触、流速可控、换热面积可设计的要求，据此可以确定计算用参数。

根据 COMSOL Multiphysics 中预热炉设计模型，炉型结构参数保持不变：

- 主筒体内径 $D = 1.2\ \text{m}$
- 有效长度 $L = 3.2\ \text{m}$
- 工作充填率 $\phi = 0.14$
- 内置 6 根导烟管，外径 $d_t = 0.168\ \text{m}$
- 预热炉主烟道半径 $r_d = 0.4\ \text{m}$

于是，主烟道流通截面积为：

$$A_d = \pi r_d^2 = \pi \times 0.4^2 = 0.503\ \text{m}^2$$

有效传热面积仍为：

$$A = \pi DL + 6\pi d_t L$$

这里第一项是筒体内壁传热面积，第二项是 6 根内置导烟管外表面积。代入数值得：

$$A = 22.20\ \text{m}^2$$

(2) 垃圾传热模型

垃圾处理过程

由模型基本设定：无论垃圾成分如何变化，预热炉均为连续进料、匀速运转，不停机。可以将基准处理规模要求 $20t/d$ 折算为连续进料质量流量：

$$\dot{m}_w = \frac{20000}{24 \times 3600} = 0.2315\ \text{kg/s}$$

由于当日垃圾必须要在当日完成处理，该量在建模中默认维持不变。以此可以进行炉内滞留量 (M_h) 和停留时间 (τ_r)的计算。

参考 公开资料，高湿度生活垃圾或中低收入地区高有机质 MSW 的典型体积密度大约在 $300–600kg/m^3$，取中间值 $450kg/m^3$ 作为设计值。

$$\varrho_b = 450\ \text{kg/m}^3$$

则炉内湿垃圾平均滞留量为：

$$M_h = \varrho_b \phi \frac{\pi D^2}{4} L$$

代入得：

$$M_h = 450 \times 0.14 \times \frac{\pi \times 1.2^2}{4} \times 3.2 = 228.0 \text{ kg}$$

于是对应平均停留时间：

$$\tau_r = \frac{M_h}{\dot{m}_w} = \frac{228.0}{0.2315} = 985.0 \text{ s} = 16.42 \text{ min}$$

这个物理量代表了垃圾在预热炉中停留时间的平均值。

垃圾物性

进炉垃圾温度可取一般环境温度：

$$T_0 = T_{amb} = 20^\circ\text{C}$$

水的比热，工程表在 20°C 时给出：

$$c_w = 4.1844 \text{ kJ/(kg} \cdot \text{K)}$$

100°C 饱和汽化潜热约为：

$$\lambda = 2257.9 \text{ kJ/kg}$$

干基垃圾比热 c_s 在已有研究中，多数落在 1.00–2.00 J/(g · K)，本模型中取偏高的中间值：

$$c_s = 1.70 \text{ kJ/(kg} \cdot \text{K)}$$

混合垃圾的等效比热正式定义为：

$$c_{eq}(x) = (1 - \omega_0(x))c_s + \omega_0(x)c_w$$

这里必须强调：正式求解中使用的是由垃圾组分实时计算得到的 $\omega_0(x)$，而不是固定的典型值 0.65。若采用 $\omega_0 = 0.65$，只能作为数量级示例或校核值，不能替代正式模型中的 $c_{eq}(x)$。

(3) 烟气传热模型

可用烟气资源

根据 COMSOL Multiphysics 中的仿真建模，主焚烧炉烟囱半径为：

$$r_s = 0.3 \text{ m}$$

于是烟囱截面积为：

$$A_s = \pi r_s^2 = \pi \times 0.3^2 = 0.2827 \text{ m}^2$$

由上一时刻等效含水率估计值 $\hat{w}_{k-1}$ 可得到当前烟囱可提供的平均温度和平均速度：

$$T_{\text{stack}}(\hat{w}_{k-1}) = -14.6746\hat{w}_{k-1} + 1446.1956$$

$$v_{\text{stack}}(\hat{w}_{k-1}) = -0.2206\hat{w}_{k-1} + 25.8437$$

从而烟囱当前可提供的最大质量流量为：

$$\dot{m}_{g,\max}^{\text{stack}} = \varrho_{\text{stack}}(\hat{w}_{k-1})A_s v_{\text{stack}}(\hat{w}_{k-1})$$

预热炉入口烟气质量流量则为：

$$\dot{m}_g = \varrho_{\text{pre}}(T_g)A_d v_g$$

因此，在考虑热胀冷缩影响时，预热炉烟气通量约束写为质量流量守恒形式：

$$\varrho_{\text{pre}}(T_g)A_d v_g \le \varrho_{\text{stack}}(\hat{w}_{k-1})A_s v_{\text{stack}}(\hat{w}_{k-1})$$

同时，预热炉可用烟气温度还必须满足：

$$T_g \le T_{\text{stack}}(\hat{w}_{k-1})$$

这里 $T_{\text{stack}}(\hat{w}_{k-1})$ 是主焚烧炉此刻实际可提供的烟气温度上限。

烟气换热系数

烟气流速对于换热系数的影响采用如下经典非线性方程：

$$U(v_g) = U_0 + kv_g^n$$

其中 n 一般取值 0.5 ~ 0.8，取中间值 $n = 0.65$。

公开工程资料里，空气/烟气冷却场景的整体传热系数常见范围约为 60–120 W/(m^2 · K)；空冷换热器的典型迎面风速大约 3 m/s；工业主风道常见速度区间大约 6–12 m/s。因此可以进行两点标定：

$$U(3) = 60, \qquad U(12) = 120$$

反解

$$U(v_g) = U_0 + kv_g^{0.65}$$

得到

$$U_0 = 18.97, \qquad k = 20.09$$

所以采用：

$$U(v_g) = 18.97 + 20.09v_g^{0.65}$$

该公式表示：烟气流速越大，换热越强，但边际提升递减。值得注意的是：v_g 需要同时满足动态烟气资源约束。

烟气物性

由于主焚烧炉烟囱温度会随垃圾含水率变化，同一质量流量的烟气在不同温度下对应的体积流量不同，因此必须考虑气体热胀冷缩，即采用质量流量守恒约束来确定烟气资源边界。

首先确定参考值。按 100–250°C 中间温区的近似烟气状态，用干空气表做一阶近似。公开热工表中，450 K（约 177°C）时干空气的定压比热约 1.0206 kJ/(kg·K)，密度约 0.7844 kg/m³；考虑真实烟气里含有 CO_2 和 H_2O，将设计值圆整为：

$$c_{pg} = 1.05 \text{ kJ/(kg} \cdot \text{K)}, \qquad \varrho_{g,\text{ref}} = 0.78 \text{ kg/m}^3$$

将烟气密度定为温度函数：

$$\varrho_g(T) = \varrho_{g,\text{ref}} \frac{T_{\text{ref},K}}{T + 273.15}$$

其中参考状态取 $T_{\text{ref},K} = 450$ K。因此有：

$$\varrho_g(T) = 0.78 \times \frac{450}{T + 273.15}$$

对应到烟囱状态和预热炉入口状态，分别有：

烟囱烟气密度：

$$\varrho_{\text{stack}}(w) = 0.78 \times \frac{450}{T_{\text{stack}}(w) + 273.15}$$

预热炉入口烟气密度：

$$\varrho_{\text{pre}}(T_g) = 0.78 \times \frac{450}{T_g + 273.15}$$

由于在近似恒压条件下，烟气温度升高，密度降低，因此即便主焚烧炉提供的烟气质量流量一定，预热炉入口允许的体积流量和流速也会随目标入口温度变化，这一点必须由约束显式体现。

(4) 预热炉的动态模型

前述传热与热负荷模型描述的是预热炉在某一给定工况下的静态热工关系，但在完整控制系统中，预热炉同样不是一个“动作立即完成、出口含水率立刻到位”的瞬时对象。当前项目中，预热炉动态模型包含三个层次的

物理过程：

1. 垃圾性质测量点到预热炉入口的前端纯延迟；
2. 炉内湿垃圾与已有物料的混合过程；
3. 设定值变化对出口含水率形成的等效慢响应。

其中，炉壁热容的影响在目前的模型中暂时忽略，因为相较于炉内垃圾的平均停留和混合过程，这一项不是主导时间尺度。

前端纯延迟

记入口垃圾组分测量点到预热炉入口的输送延迟为：

$$\tau_x = 5 \text{ s}$$

则在时间上，真正进入预热炉入口的物料性质应写为：

$$x_{\text{pre,in}}(t) = x_{\text{in}}(t - \tau_x) = x_{\text{in}}(t - 5)$$

这意味着当前测到的入口垃圾组分，并不是此刻立即参与预热过程的物料，而是 5 秒后才真正到达预热炉入口。

炉内混合的一阶等效

当前项目将预热炉内部的垃圾混合过程第一版近似为一个连续搅拌容器型的一阶混合环节。设某个等效物料性质为 $\xi(t)$，则有：

$$M_h \frac{d\xi(t)}{dt} = \dot{m}_w \, \xi_{\text{in}}(t) - \dot{m}_w \, \xi(t)$$

两边同除以 $\dot{m}_w$，可得：

$$\frac{M_h}{\dot{m}_w} \frac{d\xi(t)}{dt} = \xi_{\text{in}}(t) - \xi(t)$$

因此可定义预热炉混合一阶时间常数为：

$$\tau_{\text{mix}} = \frac{M_h}{\dot{m}_w}$$

由前文已经计算得到：

$$M_h = 228.0 \text{ kg}, \qquad \dot{m}_w = 0.2315 \text{ kg/s}$$

于是：

$$\tau_{\text{mix}} = \frac{228.0}{0.2315} = 985.0 \text{ s} = 16.42 \text{ min}$$

这说明在动态模型中，预热炉内部物料混合的主导时间尺度正是由设备尺寸、充填率和处理量共同决定的平均停留时间。

预热炉动态壳

在忽略炉壁热容、保留前端延迟和炉内混合的近似下，预热炉可以抽象为一个“5 秒纯延迟 + 985 秒一阶混合”的动态壳。对未来槽位等效物料状态 $p_{\text{slot}}(t)$，可写成：

$$\tau_{\text{mix}} \, \dot{p}_{\text{slot}}(t) = p_{\text{pre,in}}(t) - p_{\text{slot}}(t), \qquad p_{\text{pre,in}}(t) = p_{\text{in}}(t - 5)$$

若只关心出口含水率的小扰动形式，则可将预热炉的第一版动态近似写为：

$$\Delta w_{\text{out}}(s) = \frac{e^{-5s}}{985s + 1} \left[K_{wT} \, \Delta T_g(s) + K_{wv} \, \Delta v_g(s) + K_{wx} \, \Delta \omega_0(s) \right]$$

其中：

- $K_{wT} < 0$：烟气入口温度升高时，出口含水率下降；
- $K_{wv} < 0$：烟气入口速度升高时，传热增强，出口含水率下降；

- $K_{wx} > 0$：入口垃圾初始含水率升高时，出口含水率上升。

6. 预热炉能耗计算

(1) 热负荷

实验给出了“降到 20% 含水率所需时间”这一组数据，可以采用线性缩放近似，将任意目标含水率的动力学时间写成：

$$\tau(T_m, x, \omega) = \tau_{20}(T_m, x) \cdot \frac{\omega_0(x) - \omega}{\omega_0(x) - 0.20}$$

其中 $\tau_{20}(T_m, x)$ 为原始 20% 含水率动力学时间，$\omega = w/100$ 为当前目标湿基含水率。

目标含水率为 $w\%$，即湿基小数形式为 $\omega = w/100$，则每千克初始湿垃圾真正蒸发掉的水量为：

$$m_{evap}(x, w) = \frac{\omega_0(x) - w/100}{1 - w/100}$$

于是单位时间所需热负荷为：

$$\dot{Q}_{req} = \dot{m}_w \left(c_{eq}(x)(T_m - T_0) + \lambda\, m_{evap}(x, w)\right)$$

即：

$$\dot{Q}_{req} = 0.2315\,(c_{eq}(x)(T_m - 20) + 2257.9 \cdot \frac{\omega_0(x) - w/100}{1 - w/100})$$

其中第一项表示将垃圾加热到等效平均干燥温度所需的显热，第二项表示将多余水分蒸发掉所需的潜热。当目标含水率越低时，蒸发水量越大，所需热负荷越高。

(2) 热平衡约束

烟气向湿垃圾提供的热量表示如下：

$$\dot{Q}_{sup} = (U_0 + k v_g^{0.65}) A (T_g - T_m)$$

代入参数后，有：

$$\dot{Q}_{sup} = (18.97 + 20.09 v_g^{0.65}) \times 22.20 \times (T_g - T_m)/1000$$

即：

$$\dot{Q}_{sup} = [0.4211 + 0.4459 v_g^{0.65}](T_g - T_m) \quad \text{kW}$$

热平衡约束为：

$$\dot{Q}_{sup} \geq \dot{Q}_{req}$$

也就是说，烟气向湿垃圾能够提供的热功率必须大于湿垃圾干燥到对应水分所需要的热功率。

（3）单位时间能耗

预热炉的单位时间能耗以烟气质量流量计：

$$P(T_g, v_g) = \dot{m}_g c_{pg} (T_g - T_{amb})$$

在考虑热胀冷缩后，$\dot{m}_g$ 写为：

$$\dot{m}_g = \varrho_{\text{pre}}(T_g) A_d v_g$$

因此：

$$P(T_g, v_g) = \varrho_{\text{pre}}(T_g) A_d v_g c_{pg} (T_g - T_{amb})$$

可以，下面是你要补到**第七节：执行器的动态模型**里的正文。我按你前面这一章的语气和详细程度来写，尽量和前文连上。

7. 执行器的动态模型

在真实工程系统中，温度调节阀门与风机都存在响应滞后，执行器不可能瞬时完成设定值切换。因此，在动态闭环模型中，需要在优化器与预热炉之间加入执行器动态环节，用以描述“理想设定值”到“实际施加输入”之间的过渡过程。

根据当前工程近似，执行器层采用两个相互独立的一阶惯性环节表示：

- 温度通道对应阀门切换，响应较慢；
- 速度通道对应风机工况切换，响应较快。

设优化器或查表器给出的命令输入为：

$$u_c(t) = \begin{bmatrix} T_{g,c}(t) \\ v_{g,c}(t) \end{bmatrix}$$

其中：

- $T_{g,c}(t)$ 表示对预热炉入口烟气温度的命令值；
- $v_{g,c}(t)$ 表示对预热炉入口烟气速度的命令值。

设执行器实际输出为：

$$u(t) = \begin{bmatrix} T_g(t) \\ v_g(t) \end{bmatrix}$$

其中：

- $T_g(t)$ 为阀门动态作用后真正到达预热炉入口的烟气温度；
- $v_g(t)$ 为风机动态作用后真正到达预热炉入口的烟气速度。

第一版工程模型中，执行器动态采用如下对角一阶系统：

$$G_{\text{act}}(s) = \begin{bmatrix} G_T(s) & 0 \\ 0 & G_v(s) \end{bmatrix}$$

其中：

$$G_T(s) = \frac{1}{3s+1}$$

$$G_v(s) = \frac{1}{s+1}$$

于是有：

$$T_g(s) = \frac{1}{3s+1} T_{g,c}(s)$$

$$v_g(s) = \frac{1}{s+1} v_{g,c}(s)$$

其中温度通道时间常数 $3\ \text{s}$用于描述阀门开度变化、烟气混合状态调整以及烟气温度切换的综合迟滞。由于温度调节涉及阀门动作和热状态建立，因此响应相对较慢。速度通道时间常数 $1\ \text{s}$ 用于描述风机转速或流量切换的综合迟滞。由于风机工况变化通常快于温度通道，因此其时间常数取更小值。

二、控制策略与 Python 实现

1. 控制策略的总体思路

根据前述数学建模，本系统不是一个“直接以炉温为控制输入、再由执行器立刻把炉温拉回目标值”的快速单层控制对象，而是一条包含多级延迟和惯性环节的慢控制链。当前系统唯一能够持续操纵的工艺通道，是通过

调节预热炉的入口烟气温度 T_g 与入口烟气速度 v_g，改变未来离开预热炉的垃圾出口含水率，再经过焚烧炉的动态对象，逐步影响焚烧炉燃烧状态。其基本作用链条可写为：

$$[T_g,\ v_g] \longrightarrow w_{\text{out}} \longrightarrow w_b \longrightarrow \begin{bmatrix} T_{\text{avg}} \\ T_{\text{stack}} \\ v_{\text{stack}} \end{bmatrix}$$

其中：

- $[T_g,\ v_g]$ 为预热炉入口烟气温度与入口烟气速度；
- w_{out} 为预热炉出口垃圾含水率；
- w_b 为进入主焚烧炉的垃圾含水率；
- T_{avg}、T_{stack}、v_{stack} 为主焚烧炉当前可直接测得的三路输出。

因此，本系统采用分层控制结构，将“动态恢复决策”和“静态工况求解”拆开处理。当前落地代码中，各模块的职责已经明确分开：

- **外层控制器（controller）**：根据焚烧炉反馈判断当前系统偏热还是偏冷，计算等效含水率状态估计，并生成原始目标含水率请求；
- **治理器（governor）**：根据当前观测量与动态预测，把原始目标请求裁剪为当前允许送入内层优化器的最终目标；
- **转译器 A（transcriber A）**：把当前入口垃圾推进成未来槽位状态，并将治理后的目标与当前资源边界组装成静态优化请求；
- **内层优化器（optimizer）**：在给定未来槽位等效性质、目标出口含水率和资源边界的前提下，求解一组静态可行且按优先级最优的预热炉设定值；
- **转译器 B（transcriber B）**：把理想静态设定映射成当前现场真正可执行的命令；
- **执行器、预热炉与焚烧炉动态模型**：共同传播真实时域响应。

按照当前代码中的数据结构定义，完整控制链条可写为：

$$y_f(k) \to \hat{\omega}_f(k) \to \omega_{\text{req}}(k) \to \omega_{\text{tar}}(k) \to p_{\text{slot}}(k) \to \text{OptimizerRequest}(k) \to [T_g^*, v_g^*] \to [T_{g,c}, v_{g,c}] \to [T_g, v_g] \to w_{\text{out}}(k)$$

其中：

- $y_f(k)$ 表示焚烧炉三路反馈；
- $\hat{\omega}_f(k)$ 表示由三路反馈反解并融合得到的等效进炉垃圾含水率状态估计；
- $\omega_{\text{req}}(k)$ 表示 controller 给出的原始目标含水率请求；
- $\omega_{\text{tar}}(k)$ 表示经 governor 裁剪后的最终目标；
- $p_{\text{slot}}(k)$ 表示未来槽位的等效物料性质；
- $\text{OptimizerRequest}(k)$ 表示送入静态优化器的请求；
- $[T_g^*, v_g^*]$ 表示理想静态最优设定；
- $[T_{g,c}, v_{g,c}]$ 表示发往执行器的命令；
- $[T_g, v_g]$ 表示执行器动态后真正进入预热炉的输入。

从结构上说，controller 与 governor 负责解决“系统现在希望往哪边调、当前又允许调到多激进”，optimizer 负责解决“如果目标已经给定，那么在当前物料和资源边界下，最好的预热炉工况是什么”。因此，优化器不是动态控制器，而是闭环系统中的静态工况求解内核。

2. 外层控制器（Controller）

（1）焚烧炉三路反馈量

由前文数学建模，焚烧炉当前可直接测得的输出共有三项：

$$y_f(t) = \begin{bmatrix} T_{\text{avg}}(t) \\ T_{\text{stack}}(t) \\ v_{\text{stack}}(t) \end{bmatrix}$$

分别表示：

- 燃烧核心区域平均温度；
- 烟囱出口平均温度；

- 烟囱出口平均速度。

在当前工程口径下，这三路量都可以被视为“进入主焚烧炉的垃圾含水率变化”在焚烧炉动态对象上的不同观测通道。它们共享相同的动态骨架：

$$\frac{e^{-5s}}{(0.223s+1)(75.412s+1)}$$

但静态增益不同，因此其多输出动态代理模型可写为：

$$\Delta y_f(s) = \frac{e^{-5s}}{(0.223s+1)(75.412s+1)} \begin{bmatrix} -13.109632 \\ -14.412237 \\ -0.215310 \end{bmatrix} \Delta w_b(s)$$

这一定义的物理意义是：当进入焚烧炉的垃圾更湿时，燃烧核心区域平均温度降低，烟囱出口平均温度降低，烟囱出口平均速度也随之变化，但这三路量都不会立刻变化，而是要先经历 5 s 前端延迟，再经过一个快一阶和一个慢一阶。因此，焚烧炉在控制意义上是一个典型的慢对象

（2）等效含水率状态估计

由于当前闭环系统真正用于内外层衔接的中间控制变量，是“进入焚烧炉的垃圾含水率”，而这一量在现场并不能直接连续测得，因此必须通过焚烧炉三路可测输出反解出一个等效含水率估计。

根据当前代码中的静态回归系数，三路代理方程分别写为：

$$T_{\text{avg}}(w) = a_1 w + b_1, \qquad a_1 = -13.109632, \qquad b_1 = 1294.871365$$

$$T_{\text{stack}}(w) = a_2 w + b_2, \qquad a_2 = -14.412237, \qquad b_2 = 1423.472316$$

$$v_{\text{stack}}(w) = a_3 w + b_3, \qquad a_3 = -0.215310, \qquad b_3 = 25.332842$$

在实现代码 `./controller/controller.py` 中，三路反解的实现方式为：

$$\hat{\omega}_1(k) = \frac{T_{\text{avg}}(k) - 1294.871365}{100 \times (-13.109632)}$$

$$\hat{\omega}_2(k) = \frac{T_{\text{stack}}(k) - 1423.472316}{100 \times (-14.412237)}$$

$$\hat{\omega}_3(k) = \frac{v_{\text{stack}}(k) - 25.332842}{100 \times (-0.215310)}$$

这里分母额外除以 100，是因为静态代理回归的自变量使用的是百分数含水率 w，而控制器内部统一使用湿基小数形式 ω。反解后，控制器会首先把三路估计分别限制在当前代码给定的状态估计边界内：

$$0.05 \le \hat{\omega}_i \le 0.95$$

随后，再按固定权重进行融合：

$$\hat{\omega}_f(k) = \mu_1 \hat{\omega}_1(k) + \mu_2 \hat{\omega}_2(k) + \mu_3 \hat{\omega}_3(k)$$

其中当前代码中已经落地的权重为：

$$\mu_1 = 0.6, \qquad \mu_2 = 0.3, \qquad \mu_3 = 0.1$$

即：

- 平均炉温反馈权重最高；
- 烟囱出口温度反馈次之；
- 烟囱出口速度反馈权重最低。

这一步的物理意义不是“精确恢复真实瞬时进炉含水率”，而是构造一个与当前焚烧炉综合状态相一致、可用于控制器反馈的等效状态量。如果 $\hat{\omega}_f(k)$ 偏高，则说明当前焚烧炉感受到的垃圾整体偏湿、偏冷；如果 $\hat{\omega}_f(k)$ 偏低，则说明当前焚烧炉感受到的垃圾整体偏干、偏热。

（3）PID 控制律与原始目标请求

在得到等效含水率状态估计 $\hat{\omega}_f(k)$ 后，当前代码中真正参与 PID 计算的误差并不是含水率误差，而是平均炉温偏差。`./controller/controller.py` 中采用的稳态目标平均炉温为：

$$T_{\text{set}} = 872.99^{\circ}\text{C}$$

因此外层温度偏差定义为：

$$e_T(k) = T_{\text{set}} - T_{\text{avg}}(k)$$

若 $e_T(k) > 0$，说明当前炉温偏低；若 $e_T(k) < 0$，说明当前炉温偏高。当前代码中已落地的 PID 参数为：

$$K_p = 0.008, \qquad K_i = 0.00005, \qquad K_d = 0.1$$

对应的连续形式可记为：

$$C_{\text{PID}}(s) = K_p + \frac{K_i}{s} + K_d s$$

而在代码实现中，它采用的是标准离散递推结构：

1. 比例项：

$$P(k) = K_p e_T(k)$$

2. 积分项：

$$I(k) = I(k-1) + e_T(k)\,\Delta t_c$$

其中控制器更新周期在当前代码中固定为：

$$\Delta t_c = 1.0\ \text{s}$$

3. 微分项：

$$D(k) = \frac{e_T(k) - e_T(k-1)}{\Delta t_c}$$

4. 合成含水率修正量：

$$\Delta\omega(k) = K_p e_T(k) + K_i I(k) + K_d D(k)$$

然后根据当前代码的符号约定：

- 炉温偏低，表示未来垃圾需要更干；
- 炉温偏高，表示未来垃圾需要更湿。

因此原始请求不是写成 $\hat{\omega}_f + \Delta\omega$，而是写成：

$$\omega_{\text{req,raw}}(k) = \hat{\omega}_f(k) - \Delta\omega(k)$$

随后再经过三层工程限制。

第一层：总边界限幅

当前代码中，原始请求总边界为：

$$0.20 \le \omega_{\text{req}} \le 0.60$$

这里 0.20 是预热炉动力学模型的最低适用出口含水率，也是当前系统不再继续向更干方向追求的总下界；0.60 则是当前控制器允许提出的最湿目标上界。

第二层：单步变化率限制

当前代码中，为避免外层请求在慢对象上过快跳变，还引入了每步变化率限制：

$$|\omega_{\text{req}}(k) - \omega_{\text{req}}(k-1)| \le 0.008$$

由于 controller 的更新周期为 1 s，这意味着原始请求的最大变化速率约为：

$$0.008\ \text{s}^{-1}$$

它的物理意义是：即便当前温度偏差很大，外层控制器也不会在一个控制周期内把目标一下子推得过猛，而是采取较平缓的目标推进方式。

第三层：输出低通滤波

在代码中，原始请求还经过一阶低通平滑：

$$\omega_{\text{req}}(k) = \beta\,\omega_{\text{req}}(k-1) + (1-\beta)\,\omega_{\text{req,limited}}(k)$$

其中当前已确定参数为：

$$\beta = 0.88$$

这意味着请求链具有较强的历史保持能力。其目的不是改变稳态方向，而是减少目标在延迟链条中的高频抖动，降低后续 governor、查表与执行器频繁切换的风险

（4）积分抗饱和与积分分离

由于系统总体上是一条长延迟、慢惯性的控制链，如果积分项在纯延迟阶段持续累积，很容易出现 windup。当前代码中已经落地的积分保护措施有两项。

其一，积分抗饱和。当前代码不是直接限制积分值本身，而是先对积分通道允许输出的含水率修正量进行限幅：

$$-0.12 \le K_i I(k) \le 0.12$$

等价地，积分内部状态被限制在：

$$\frac{-0.12}{0.00005} \le I(k) \le \frac{0.12}{0.00005}$$

即：

$$-2400 \le I(k) \le 2400$$

这说明当前设计允许积分项在较大范围内累积，但不会无限扩大。比例项和微分项负责快速修正，积分项只承担缓慢拉回稳态偏差的作用。

其二，积分分离。当前代码中，积分分离是开启的：

$$\text{enable_integral_separation} = \text{True}$$

但其死区参数当前设置为：

$$\text{integral_deadband_C} = 0$$

因此在现阶段代码里，只有当误差恰好为零时才会暂停积分。这意味着从逻辑上积分分离接口已经留出，但在当前参数下它几乎退化为“始终允许积分”。后续若闭环测试发现低频 hunting 仍然明显，可以优先调大这一死区。

综上，当前 controller 的输出不是执行器命令，而是一个经过状态估计、PID 修正、变化率限制和低通滤波后的原始目标请求：

$$\omega_{\text{req}}(k)$$

这个量代表了外层控制器围绕稳态目标所提出的“未来垃圾应更湿还是更干”的控制意图。

3. 治理器（Governor）

由外层控制器得到的 $\omega_{\text{req}}(k)$ 仍然只是一个“希望如此”的原始意图，并不能直接送入内层优化器。原因在于：

1. 稳态工作点只描述长期运行最适宜的参考位置，并不意味着任意时刻都必须死守在稳态窄带内；
2. 当前系统已经明确把“稳态约束”与“动态允许目标”重新分层，在线动态闭环推荐使用 `advisory` 优化器，而不再让优化器替动态系统做稳态审判；
3. 即使外层提出了某个目标，该目标也未必在当前动态阶段和可测输出约束下合理；

4. 若不设置治理器，外部控制器会在一个慢对象上直接与静态优化器耦死，使目标决策与工况求解职责混乱。

因此，在外层控制器与内层优化器之间需要设置目标治理模块。当前代码中的治理器并不是从稳态带机械派生动态带，而是基于当前可测量量，尤其是当前 T_{avg}，在线生成动态允许带。其主要配置参数已经在 `./governor/governor.py` 中固定如下：

$$\omega_{ss,\min} = 0.3042675804697915, \qquad \omega_{ss,\max} = 0.3393469511577442$$

表示稳态参考带的上下界；

$$\omega_{\text{dyn,min,cap}} = 0.20, \qquad \omega_{\text{dyn,max,cap}} = 0.60$$

表示治理器总允许边界；

$$T_{\text{avg,dyn,min}} = 845.0^{\circ}\text{C}, \qquad T_{\text{avg,dyn,max}} = 1105.0^{\circ}\text{C}$$

表示当前治理器用来做动态监督的温度区间；

$$H = 600\ \text{s}, \qquad \tau_{\text{eff}} = 1060\ \text{s}$$

表示动态预测视界与等效总响应时间常数；

$$\beta_{\text{dyn}} = 0.80$$

表示动态带本身的一阶平滑系数。

（1）动态响应因子

当前治理器首先计算一个短中期响应因子：

$$\phi(H) = 1 - e^{-H/\tau_{\text{eff}}}$$

代入当前代码中已经确定的参数：

$$\phi(600) = 1 - e^{-600/1060} \approx 0.432$$

这表示：治理器并不假设“当前目标一改变，焚烧炉很快就能完全响应”，而只假设在未来约 600 s 内，系统只能实现大约 43.2% 的等效响应。这个设计直接把对象慢响应的事实纳入了动态允许带生成逻辑。

（2）由当前测量量反推允许目标带

治理器使用当前观测到的 T_{avg} 与当前融合后的 ω_{est}，近似计算在未来视界 H 内允许达到的目标含水率区间。由于：

$$T_{\text{avg}}(w) = 100\, a\, \omega + b, \qquad a = -13.109632$$

所以治理器使用的线性化有效系数为：

$$k = 100 \times a \times \phi(H)$$

代入当前参数，有：

$$k \approx 100 \times (-13.109632) \times 0.432 \approx -566.3$$

再结合动态监督温度边界，反推出当前允许的含水率带：

$$\omega \in [\omega_{\text{est}} + \frac{T_{\text{avg,dyn,max}} - T_{\text{avg}}}{k},\ \omega_{\text{est}} + \frac{T_{\text{avg,dyn,min}} - T_{\text{avg}}}{k}]$$

随后再与治理器总边界 $[0.20, 0.60]$ 取交。由于 $k < 0$，在代码实现中会自动重新排序上下界，得到基于当前 T_{avg} 的动态允许区间。

（3）稳态参考带的并入与动态带平滑

当前治理器并不会把稳态参考带硬塞给优化器，但也不会完全丢掉稳态参考带。代码中的处理方式是：

- 先得到由当前测量量反推的动态带 $[\omega_{lo}^{Tavg}, \omega_{hi}^{Tavg}]$；

- 再与稳态参考带 $[\omega_{ss,\min}, \omega_{ss,\max}]$ 做并包；
- 最后对动态带本身做一阶平滑。

原始动态带写为：

$$\omega_{\text{lo,raw}} = \max\{0.20,\ \min(\omega_{lo}^{T_{avg}}, \omega_{ss,\min})\}$$

$$\omega_{\text{hi,raw}} = \min\{0.60,\ \max(\omega_{hi}^{T_{avg}}, \omega_{ss,\max})\}$$

然后使用带宽平滑：

$$\omega_{\text{lo}}(k) = 0.80\,\omega_{\text{lo}}(k-1) + 0.20\,\omega_{\text{lo,raw}}(k)$$

$$\omega_{\text{hi}}(k) = 0.80\,\omega_{\text{hi}}(k-1) + 0.20\,\omega_{\text{hi,raw}}(k)$$

从而得到最终动态允许目标带：

$$W_{\text{dyn}}(k) = [\omega_{\text{lo}}(k), \omega_{\text{hi}}(k)]$$

最后，原始请求被投影为：

$$\omega_{\text{tar}}(k) = \Pi_{W_{\text{dyn}}(k)}(\omega_{\text{req}}(k))$$

同时治理器还会给出一个基于当前目标的短中期平均炉温预测值：

$$T_{\text{avg,pred}}(k) = T_{\text{avg}}(k) + 100\,a\,[\omega_{\text{tar}}(k) - \omega_{\text{est}}(k)]\,\phi(H)$$

这一预测值当前主要作为诊断输出，帮助理解动态治理器是否在把目标推向一个合理的方向。

因此，治理器的本质可以概括为：

- controller 决定“系统希望往哪边调”；
- governor 决定“当前最多允许调到哪里”；
- optimizer 只在治理后的目标上求静态最优设定。

4. 转译器 A（Transcriber A）

在这套系统中，当前测得的入口垃圾组分 $x_{\text{in}}(t)$ 并不是当前被控对象。因为该物料还要经历：

1. 测量点到预热炉入口的前端纯延迟；
2. 预热炉内的长时间混合与停留；
3. 预热炉出口到焚烧炉入口的后续输送；
4. 焚烧炉本体的延迟与惯性。

因此，外层和内层真正应当面对的不是当前刚测到的垃圾，而是未来某一时刻将从预热炉出口出来并最终影响焚烧炉的那一槽位等效物料。这就是未来槽位的含义。

当前代码中，转译器 A 的更新周期已经固定为：

$$\Delta t_{TA} = 2.0\ \text{s}$$

它的前端延迟参数为：

$$\tau_x = 5.0\ \text{s}$$

混合时间常数为：

$$\tau_{\text{mix}} = 985.0\ \text{s}$$

并固定使用：

$$\text{burn_policy} = \text{"}advisory\text{"}$$

也就是说，当前转译器 A 在组装静态优化请求时，已经明确按在线动态闭环推荐语义把优化器放在 `advisory` 模式下。

（1）由清洗器（cleanser）生成入口等效性质

转译器 A 首先调用 `cleanser.cleanser`，把 6 维组分比例转换为三维等效性质：

$$p_{\text{in}}(k) = \begin{bmatrix} \omega_{0,\text{in}}(k) \\ t_{\text{ref,in}}(k) \\ s_{\text{in}}(k) \end{bmatrix}$$

其中：

- $\omega_{0,\text{in}}$ 为入口初始湿基含水率；
- $t_{\text{ref,in}}$ 为 175°C 下达到 20% 含水率所需时间；
- s_{in} 为温度敏感性斜率。

（2）前端延迟离散实现

在连续模型里，前端纯延迟写成：

$$p_{\text{pre,in}}(t) = p_{\text{in}}(t-5)$$

而在当前代码里，transcriber A 用一个长度为 `delay_steps` 的队列来离散实现这件事：

$$N_d = \text{round}\,(\frac{5.0}{2.0}) = 2$$

因此，当前代码实际使用的是一个 2 个更新步长的延迟队列，对应约 4 s 的离散前端延迟。它的物理目标仍然是 5 s 延迟，但由于当前更新周期取 2 s，代码中使用 `round` 后会得到 2 步。也就是说，这一离散化在工程意义上是“按 5 s 目标建模，但在当前步长下近似实现为 4 s 延迟”。后续若需要更精确地贴近 5 s，可将更新周期继续细化，或把离散方式改为 `ceil`。

（3）一阶混合槽位更新

当前代码中，未来槽位状态使用 985 s 一阶混合模型推进。连续形式为：

$$\tau_{\text{mix}}\dot{p}_{\text{slot}}(t) = p_{\text{pre,in}}(t) - p_{\text{slot}}(t)$$

在离散化后，代码中使用：

$$\alpha = e^{-\Delta t_{TA}/\tau_{\text{mix}}} = e^{-2/985} \approx 0.99797$$

于是槽位状态递推为：

$$p_{\text{slot}}(k+1) = \alpha p_{\text{slot}}(k) + (1-\alpha)p_{\text{delayed}}(k)$$

这里的 $p_{\text{delayed}}(k)$ 是队列中已经经过前端延迟的入口等效性质。由于 α 非常接近 1，这反映了预热炉内部是一个极慢的混合过程：在 2 s 的更新尺度上，槽位状态只会缓慢地向新的入口物料靠近。

（4）资源边界与静态请求组装

在未来槽位状态更新后，转译器 A 会进一步组装优化器请求：

$$\text{OptimizerRequest}(k) = (\omega_0^{\text{slot}},\ t_{\text{ref}}^{\text{slot}},\ s^{\text{slot}},\ \omega_{\text{tar}}(k),\ r(k),\ \text{policy}(k))$$

在当前代码中，如果现场资源边界没有显式输入，则采用默认资源上限：

$$T_{\text{stack,cap}} = 930.0^{\circ}\text{C}$$

$$v_{\text{stack,cap}} = 18.0\ \text{m/s}$$

这两个默认值用于构造 `ResourceBoundary`，随后再把 governor 输出的 `dyn_band` 一并包装进 `DynamicTargetBand`。因此，当前 transcriber A 的输出，不是执行器命令，而是一个明确面向静态优化器的数据请求包。

5. 内层优化器（Optimizer）

内层优化器是当前系统中最完整、最成熟的模块之一。它不是动态控制器，而是一个静态优化映射。其功能可以概括为：

> 在给定未来槽位物料性质、目标出口含水率、当前可用烟气资源边界以及动态允许目标带的前提下，求解一组静态可行且按优先级最优的预热炉设定值。

（1）输入字段与语义

当前代码中，优化器请求结构 `OptimizerRequest` 包含以下字段：

1. `props: EquivalentProperties`，即未来槽位等效物料性质，包含：
   - `omega0`：入口初始湿基含水率；
   - `tref_min`：175°C 下达到 20% 含水率所需时间；
   - `slope_min_per_c`：温度敏感性斜率。
2. `omega_tar`：治理器给出的最终目标出口含水率；
3. `resource: ResourceBoundary`：当前可用烟气资源边界，包含：
   - `T_stack_cap_C`：当前可用烟气温度上限；
   - `v_stack_cap_mps`：当前可用烟气速度上限；
4. `dyn_band: DynamicTargetBand`：当前动态允许目标带，包含：
   - `omega_min`；
   - `omega_max`；
5. `slot_id`：槽位标识；
6. `burn_policy`：当前求解策略，当前在线动态闭环固定采用：

$$\text{burn_policy} = \text{"}advisory\text{"}$$

   `strict` 只保留给离线筛选、模型验证和极保守场景。

（2）决策变量

当前优化器内部的决策向量为：

$$z = [T_g \quad v_g \quad T_m \quad \omega \quad d_\omega^- \quad d_\omega^+]^\top$$

其中：

- T_g：预热炉入口烟气温度；
- v_g：预热炉入口烟气速度；
- T_m：垃圾层等效平均干燥温度；
- ω：预热后出口垃圾目标含水率；
- d_ω^-、d_ω^+：目标偏差松弛变量。

当前代码中的静态边界为：

$$100 \le T_g \le \min(T_{\text{stack,cap}}, 2000)$$

$$3 \le v_g \le 12$$

$$20 \le T_m \le 250$$

而 ω 的边界并不是直接取某个固定常数区间，而是先由动态允许带与物理可实现带求交后得到。

（3）目标带、物理可实现带与目标投影

当前优化器在正式求解前，首先会把治理器给出的动态允许目标带，与物理可实现带求交。代码中的物理可实现带定义为：

- 下界不能低于动力学模型定义的 20%：

$$\omega \ge 0.20$$

- 上界不能高于入口初始含水率，因为预热炉不会“加水”：

$$\omega \le \omega_0$$

因此，实际送入优化器的可行目标带为：

$$[\omega_{\min}^{\text{applied}}, \omega_{\max}^{\text{applied}}] = W_{\text{dyn}}(k) \cap [0.20, \omega_0]$$

若治理器提出的目标落在该区间之外，优化器还会进一步做目标投影：

$$\omega_{\text{tar,projected}} = \Pi_{[\omega_{\min}^{\text{applied}},\omega_{\max}^{\text{applied}}]}(\omega_{\text{tar}})$$

这说明治理器管的是当前动态允许目标，优化器能测定当前目标是否具有物理可行性。即使治理器允许某个目标，若该目标超出了当前物料物理可实现边界，优化器仍然会继续把目标投影回物理可实现带。

（4）热工基础函数

根据数学建模中的内容，当前优化器内部使用以下热工函数：

等效比热

$$c_{eq} = (1-\omega_0)c_s + \omega_0 c_w$$

其中代码参数为：

$$c_s = 1.70\ \text{kJ/(kg}\cdot\text{K)}, \qquad c_w = 4.1844\ \text{kJ/(kg}\cdot\text{K)}$$

蒸发水量

$$m_{\text{evap}} = \frac{\omega_0 - \omega}{1-\omega}$$

仅当 $\omega_0 > \omega$ 时为正。

供热能力

优化器使用的供热能力为：

$$\dot{Q}_{\text{sup}} = (U_0 + K_U v_g^n)A(T_g - T_m)/1000$$

其中当前参数为：

$$U_0 = 18.97, \qquad K_U = 20.09, \qquad n = 0.65, \qquad A = 22.20\ \text{m}^2$$

所需热负荷

$$\dot{Q}_{\text{req}} = \dot{m}_w(c_{eq}(T_m - T_0) + \lambda m_{\text{evap}})$$

其中参数为：

$$\dot{m}_w = \frac{20000}{86400} = 0.2315\ \text{kg/s}$$

$$T_0 = 20^\circ\text{C}, \qquad \lambda = 2257.9\ \text{kJ/kg}$$

预热动力学时间

当前优化器沿用线性缩放近似：

$$\tau_{20}(T_m, x) = t_{\text{ref}} + s(T_m - 175)$$

$$\tau_{\text{target}}(T_m, x, \omega) = \tau_{20}(T_m, x) \cdot \frac{\omega_0 - \omega}{\omega_0 - 0.20}$$

单位时间能耗

$$P(T_g, v_g) = \varrho_g(T_g)A_D v_g c_{pg}(T_g - T_{amb})$$

其中参数为：

$$c_{pg} = 1.05, \qquad T_{amb} = 20^\circ\text{C}, \qquad A_D = \pi \times 0.4^2$$

并采用温度相关烟气密度模型：

$$\varrho_g(T) = 0.78 \times \frac{450}{T + 273.15}$$

这些函数共同构成优化器静态求解的热工基础。

（5）约束条件的完整结构

优化器的由多类约束共同组成，可分为以下几类。

目标跟踪等式约束

$$\omega + d_\omega^- - d_\omega^+ = \omega_{\text{tar,projected}}$$

该式表示在允许存在小偏差的情况下，尽量使优化器输出含水率贴近治理后的目标含水率。

停留时间约束

$$\tau_r - \tau_{\text{target}}(T_m, x, \omega) \geq 0$$

其中当前代码中的平均停留时间为：

$$\tau_r = \frac{M_H}{\dot{m}_w} = \frac{228.0}{0.2315} = 985 \text{ s} = 16.42 \text{ min}$$

该约束表示：在当前设定下，垃圾在炉内平均停留时间必须足够完成预期脱水。

供热能力约束

$$\dot{Q}_{\text{sup}} - \dot{Q}_{\text{req}} \geq 0$$

该约束表示：预热炉的实际供热能力必须不小于垃圾达到目标含水率所需的热负荷。

烟气资源质量流量约束

$$\dot{m}_{\text{stack,cap}} - \dot{m}_{\text{preheater}} \geq 0$$

其中：

$$\dot{m}_{\text{stack,cap}} = \varrho_g(T_{\text{stack,cap}}) A_s v_{\text{stack,cap}}$$

$$\dot{m}_{\text{preheater}} = \varrho_g(T_g) A_D v_g$$

它表示：当前预热炉入口所需烟气质量流量不能超过主焚烧炉当前可提供的烟气资源上限。

最小传热驱动力约束

$$T_g - T_m \geq 0$$

当前代码中 `DELTA_T_MIN = 0`，即只要求传热方向不反转，不额外强迫保留正安全裕度。

不加水约束

$$\omega_0 - \omega \geq 0$$

即：预热炉只会把垃圾变干，不会把垃圾变得比入口更湿。

严格稳燃约束（仅 strict 模式启用）

若 `burn_policy="strict"`，优化器还会额外加入：

$$T_{\min}(w) \geq 850$$

$$T_{\text{avg}}(w) \geq 850$$

$$T_{\text{avg}}(w) \leq 1100$$

$$T_{\max}(w) \leq 1100$$

当前在线动态闭环明确推荐 `advisory`，因此在正常在线运行中，这些焚烧炉稳态代理量主要作为诊断输出，而不是优化器的硬边界。

（6）三阶段词典序多目标求解

当前优化器使用的是三阶段 SLSQP 词典序求解，而不是把所有目标简单线性加权。其含义是：不同目标之间不是“谁权重大一点”，而是有明确的先后优先级。

第一阶段：目标含水率跟踪优先

首先最小化：

$$P_1 = d_\omega^- + d_\omega^+$$

这一步是在满足所有硬约束的前提下，尽可能让输出含水率靠近治理器给出的目标。

第二阶段：燃烧均匀性优先

在第一阶段最优值保持不变（允许一个很小容差）的前提下，再最小化：

$$P_2 = \sigma(w)$$

其中：

$$\sigma(w) = -0.189997w + 18.331239$$

这表示：在跟踪目标已经尽量好的前提下，再进一步追求更均匀的燃烧温度分布。

第三阶段：能耗最小

在前两阶段最优值都保持不变（允许一个很小容差）的前提下，再最小化：

$$P_3 = P(T_g, v_g)$$

即在控制性能已满足的情况下，再追求最低能耗。

当前代码的具体做法是：

1. 第一阶段先全种子搜索，求出 P_1^*；
2. 第二阶段把 $P_1 \le P_1^* + \varepsilon$ 加为额外不等式，再求 P_2^*；
3. 第三阶段同时保持 $P_1 \le P_1^* + \varepsilon$ 与 $P_2 \le P_2^* + \varepsilon$，再最小化 P_3。

这套词典序结构正是运筹学手册中“先稳燃与跟踪，再均匀性，最后能耗”的工程思想在当前代码中的直接落地。

（7）初值种子与数值求解设置

为了提高求解鲁棒性，当前优化器并不是只用一组初值直接做 SLSQP，而是使用多组物理启发式初值。当前代码中的初值候选主要来自：

- T_g 候选：

$$T_g \in \{100,\ \min(300, T_g^{\max}),\ T_g^{\max}\}$$

- v_g 候选：

$$v_g \in \{3.5,\ 8.0,\ 11.0\}$$

- T_m 候选：

$$T_m \in \{55,\ 90\}$$

- ω 候选：

$$\omega \in \{\omega_{\min}^{\text{band}},\ \omega_{\text{tar,projected}},\ \omega_{\max}^{\text{band}}\}$$

这些种子组合形成多组初始点，优化器会逐一尝试，以降低局部失败的概率。当前代码中的数值设置还包括：

$$\text{MAXITER} = 120$$

$$\text{STAGE_TOL} = 10^{-6}$$

$$\text{FEAS_TOL} = 10^{-2}$$

$$\text{BIG_SLACK} = 10^{6}$$

其中：

- `MAXITER` 控制 SLSQP 最大迭代步数；
- `STAGE_TOL` 用于控制词典序阶段间允许的数值容差；
- `FEAS_TOL` 是最终可行性的判定阈值；
- `BIG_SLACK` 是偏差变量的数值上界。

（8）输出字段与诊断量

在求解完成后，当前优化器输出的主要工艺变量包括：

$$T_g^*, \quad v_g^*, \quad T_m^*, \quad \omega^{\mathrm{opt}}$$

同时还会给出大量诊断量，包括：

- `omega_tar_requested`：原始目标；
- `omega_tar_projected`：经过物理与动态带投影后的目标；
- `omega_band_applied_min/max`：实际施加的目标带；
- `d_omega_minus / d_omega_plus`：目标偏差；
- `tau20_min / tau_target_min / tau_r_min`：动力学时间信息；
- `Qsup_kW / Qreq_kW`：供热能力与热负荷；
- `mdot_stack_cap_kgps / mdot_preheater_kgps`：烟气资源占用；
- `Tavg_burn_C / Tmin_burn_C / Tmax_burn_C / sigma_burn_C`：焚烧炉稳态代理诊断量；
- `steady_band_violation_percent`：与稳态参考带的偏离程度；
- `feasible_under_strict_burn`：若切换到 strict 模式时是否仍稳燃可行；
- `stage1_objective / stage2_objective / stage3_objective`：三阶段目标函数值；
- `max_constraint_violation`：最大约束违背程度；
- `message`：当前求解语义说明。

因此，当前 optimizer 并不仅仅输出一个“查表结果”，而是同时输出一整套可用于工程诊断、离线分析和在线记录的约束核验信息。这也是它当前已经高度工程化的重要体现。

（9）离线制表与在线查表

当前系统已经启用了离线制表与在线查表的实现路线。当前主表轴为：

$$(\omega_0,\ t_{\mathrm{ref}},\ s,\ \omega_{\mathrm{tar}})$$

即主表只围绕：

- 未来槽位入口初始含水率；
- 参考干燥时间；
- 温度敏感性斜率；
- 最终目标含水率

四个维度构造。资源边界与动态允许带不作为主表轴，而在在线端由治理器和转译器 B 进行修正。这一分层使得主表可以继续使用，而不必因为动态治理逻辑重构而全部重制。

在线端则不再实时求解非线性规划，而是只保留：

- 目标计算；
- future slot 推进；
- 查表或插值；
- 在线资源修正。

因此，优化器能够在不改变数学内核的前提下无痛接入整个动态闭环系统：它仍然是静态求解器，只是其输入不再是“当前入口垃圾”，而是“未来槽位垃圾”；其输出也不再被误解为“炉子下一瞬间会达到的状态”，而是“从当前开始施加、将对未来槽位产生作用的理想静态设定”。

6. 转译器 B（Transcriber B）

优化器查表或静态求解输出的 T_g^* 与 v_g^*，只是理想静态最优设定，还不是当前时刻可以直接送入执行器的最终命令。因为现场当前还受到以下限制：

1. 当前烟囱可提供的温度上限；
2. 当前烟囱可提供的质量流量上限；
3. 现场是否允许启用补热；
4. 当前命令变化速度是否过快。

因此，需要由转译器 B 对理想设定做在线修正。设优化器输出的理想静态设定为：

$$u^*(k) = \begin{bmatrix} T_g^*(k) \\ v_g^*(k) \end{bmatrix}$$

则转译器 B 根据当前烟囱资源边界，生成实际命令：

$$u_c(k) = \begin{bmatrix} T_{g,c}(k) \\ v_{g,c}(k) \end{bmatrix}$$

（1）命令变化率限制

当前 `transcriber_b.py` 先对理想命令做变化率限制。当前代码参数为：

$$\Delta T_{g,\max} = 40^\circ\mathrm{C/step}$$

$$\Delta v_{g,\max} = 0.8\ \mathrm{m/s/step}$$

由于 transcriber / optimizer 层当前更新周期为 2 s，因此这相当于：

$$20^\circ\mathrm{C/s}$$

$$0.4\ \mathrm{m/s^2}$$

这一步的物理意义是：即便查表结果在相邻刷新周期之间变化较大，转译器 B 也不会把这种跳变原样送给执行器，而会先把命令变化速度限制在现场更容易接受的范围内。

（2）设备基本边界

变化率限制后，代码继续施加设备基本边界。当前参数为：

$$100 \le T_g \le 2000$$

$$3 \le v_g \le 12$$

若命令触及这些边界，则同时打上诊断标记：

- `resource_limited`
- `Tg_limited_by_stack`
- `vg_limited_by_stack`

（3）烟气资源修正

随后，transcriber B 会用当前命令重新计算预热炉所需烟气质量流量：

$$\dot{m}_{\mathrm{need}} = \varrho_g(T_g) A_D v_g$$

并与优化器输出的当前烟囱资源上限进行比较。若：

$$\dot{m}_{\mathrm{need}} > \dot{m}_{\mathrm{cap}}$$

则判定当前命令超出了当前烟囱可提供的质量流量，此时会进一步根据资源上限反算速度上限：

$$v_{g,\mathrm{cap}} = \frac{\dot{m}_{\mathrm{cap}}}{\varrho_g(T_g) A_D}$$

并令：

$$v_{g,c} = \min(v_{g,c}, v_{g,\mathrm{cap}})$$

这意味着：转译器 B 的在线修正不是简单限幅，而是基于当前温度命令、当前密度和当前可用烟气质量流量上限，重新推导一个物理上可实现的速度命令。

（4）补热缺口计算

在速度与温度命令都被资源修正后，代码还会进一步估算当前命令所能提供的实际可用热功率：

$$Q_{\text{available}} = \dot{m}_{\text{actual}}\, c_{pg}\, (T_g - T_{amb})$$

若优化器所需热负荷为 Q_{req}，则热缺口定义为：

$$Q_{\text{heat,deficit}} = \max(Q_{\text{req}} - Q_{\text{available}}, 0)$$

若该缺口大于零，则：

- `heater_enable = True`
- `Q_heat_deficit_kW = Q_{\mathrm{heat,deficit}}`

因此，转译器 B 并不是“重新优化器”，而是理想静态设定到现场执行命令之间的在线投影器。它使得优化器始终可以保持静态求解职责不变，同时确保现场实际命令符合当前真实资源边界。

7. 闭环动态过程的完整说明

综上，完整闭环中的动态过程可以明确描述如下。

当焚烧炉偏热时

1. 焚烧炉三路反馈量 T_{avg}、T_{stack}、v_{stack} 整体升高；
2. controller 通过三路稳态反解和加权融合，得到较低的 $\hat{\omega}_f$，判断当前焚烧炉所对应的进炉垃圾整体偏干、偏热；
3. 由误差 $e_T = T_{\text{set}} - T_{\text{avg}}$ 得到负向 PID 修正，使得 ω_{req} 升高；
4. governor 检查当前动态监督边界，若该请求过激，则把它裁剪成 ω_{tar}；
5. transcriber A 将当前入口垃圾经过前端延迟与一阶混合，推进成未来槽位状态；
6. optimizer 接收未来槽位性质、治理后的最终目标和当前资源边界，求出理想静态设定 T_g^*、v_g^*；
7. transcriber B 再将这组理想静态设定投影成当前可执行命令 $T_{g,c}$、$v_{g,c}$，必要时限幅并给出补热请求；
8. 执行器按一阶惯性逐步跟踪命令；
9. 预热炉在“前端延迟 + 一阶混合”的动态壳中逐步形成更湿的出口垃圾；
10. 更湿的垃圾进入焚烧炉后，再经过焚烧炉“5 s 纯延迟 + 双一阶惯性”逐步把平均温度拉回稳态。

当焚烧炉偏冷时

过程完全相反：

- controller 会提出“未来垃圾更干”的原始请求；
- governor 会判断当前动态阶段允许把目标推进到什么程度；
- optimizer 会给出更强的预热设定；
- 未来出口含水率逐步下降；
- 焚烧炉再经过延迟与惯性逐步升温。

因此，当前系统绝不能被理解为“测到炉温变化后下一秒就能把炉温拉回去”的快控制器，而应被理解为一条通过未来垃圾含水率间接影响焚烧炉温度的慢控制链。

8. 在线实现与时间尺度

当前工程的时间尺度已经不再只是概念建议，而是代码中已经基本确定的实际实现值。

（1）测量层

在 `tests/sim_common.py` 中，焚烧炉测量与仿真层的步长已经固定为：

$$\Delta t_{\text{meas}} = 0.1\ \text{s}$$

即 10 Hz 采样。该层负责：

- 获取焚烧炉原始三路反馈；
- 推进预热炉和焚烧炉动态对象；
- 记录连续轨迹与时域响应。

在这一层上，几个关键延迟与惯性折算为：

- 预热炉前端延迟：

$$\frac{5.0}{0.1} = 50 \text{ 个测量步}$$

- 焚烧炉前端延迟：

$$\frac{5.0}{0.1} = 50 \text{ 个测量步}$$

- 预热炉混合时间常数：

$$985 \text{ s} = 9850 \text{ 个测量步}$$

- 焚烧炉慢时间常数：

$$75.412 \text{ s} = 754.12 \text{ 个测量步}$$

- 焚烧炉快时间常数：

$$0.223 \text{ s} = 2.23 \text{ 个测量步}$$

这说明测量层确实是“看得快”，但对象本身响应非常慢。

（2）外层控制层

当前 `./controller/controller.py` 与 `./governor/governor.py` 中都已经把控制更新周期固定为：

$$\Delta t_{\text{ctrl}} = \Delta t_{\text{gov}} = 1.0 \text{ s}$$

这意味着：

- controller 每 1 s 更新一次等效状态估计与原始目标请求；
- governor 每 1 s 更新一次动态允许带与最终目标。

按这一尺度，焚烧炉前端 5 s 延迟约对应：

$$\frac{5.0}{1.0} = 5 \text{ 个控制步}$$

也就是说，外层控制器在 5 个更新周期内都还看不到自己的动作完全传递到焚烧炉入口，因此它必须采取“目标缓慢推进”的方式，而不能高频猛调。

（3）转译与优化层

当前 `transcriber_a.py` 中已经把更新时间固定为：

$$\Delta t_{\text{opt}} = 2.0 \text{ s}$$

这意味着：

- future slot 每 2 s 更新一次；
- optimizer / lookup table 每 2 s 刷新一次输入；
- transcriber B 也按这一节奏生成新的命令。

按这一尺度，预热炉前端 5 s 延迟在当前代码里被离散成约 2 个更新步，即约 4 s；而 985 s 混合时间常数约对应：

$$\frac{985}{2} = 492.5 \text{ 个优化更新步}$$

这进一步说明：内层优化器面对的是一个极慢变化的 future slot 静态问题，而不是高速在线动态优化问题。

（4）执行器层

执行器本体动态来自前文数学建模：

- 温度通道时间常数：

$$\tau_T = 3\ \mathrm{s}$$

- 速度通道时间常数：

$$\tau_v = 1\ \mathrm{s}$$

而在命令层，transcriber B 还进一步引入了单步变化率限制：

- 每 2 s 最多改变 $40^\circ\mathrm{C}$；
- 每 2 s 最多改变 $0.8\ \mathrm{m/s}$。

因此，当前工程实际上采用的是三层时间结构：

1. **0.1 s 测量与动态仿真层**：负责“看得快”；
2. **1 s controller / governor 层**：负责“想得稳”；
3. **2 s transcriber / optimizer / 命令层**：负责“算得实、做得稳”。

三、下一步计划

- ▢基于当前的控制策略及其 Python 实现，开发垃圾站监控客户端 APP；
- ▢将当前外部控制器控制策略由 PID 升级为 Smith-PID，以适应大延迟场景；
- ▢将当前的控制策略于 MATLAB/Simulink 中进行完整建模与仿真验证；
- ▢优化器重新制表并使用数据库进行优化
- ▢将当前的控制策略于微控制器中进行实现
- ▢投稿期刊